

Contents

A Reproducible Runtime Performance Benchmark Across Ruby, Go, Python, and Node.js (via Background Job Execution)	1
Abstract	1
1. Introduction	1
2. Related Work	2
3. Experimental Design	2
4. Implementation Overview	4
5. Data Collection & Export	5
6. Visualization	5
7. Statistical Treatment	5
8. Results	6
9. Discussion	7
10. Reproducibility	8
11. Limitations & Threats to Validity	11
12. Conclusion & Future Work	12
13. References	12

A Reproducible Runtime Performance Benchmark Across Ruby, Go, Python, and Node.js (via Background Job Execution)

Abstract

We present an empirical comparison of four contemporary ecosystems for background job processing and computation-heavy tasks: Ruby on Rails with Sidekiq, Go with a Sidekiq-compatible Redis consumer, Python with Celery (with a small Flask enqueue bridge), and Node.js with a Sidekiq-compatible Redis consumer. Using a unified PostgreSQL schema, identical data structures (arrays / hashes / objects) and minimized framework abstractions inside workers, we measure task duration distributions, memory usage, and throughput across varying dataset slice sizes (**per_page**). We contribute: (1) a reproducible multi-language benchmark harness; (2) standardized wide and long-format CSV exports for raw result series; (3) D3-based visualizations of distributional statistics; and (4) guidance on technology selection under different latency / resource profiles. Here, reproducibility means deterministic data generation, fixed orchestration, and a documented environment; repeated benchmark runs on a real machine are expected to yield statistically similar measurements rather than byte-identical exported artifacts. All code and data are open-source for verification and extension.

Artifact availability: <https://github.com/al1ejandro/language-performance-benchmark-harness> (includes Docker Compose definitions, benchmark orchestration tasks, the generated CSV/SVG artifacts under **benchmark_ui/docs/**, and the benchmark UI plus all worker implementations in one archival repository snapshot).

1. Introduction

Background job processing underpins web application responsiveness and scalability. While prior informal comparisons focused on small subsets of languages, engineering teams today routinely evaluate multiple ecosystems when optimizing computational workloads (e.g., aggregation, statistical

transforms, ETL). This investigation delivers a controlled, reproducible cross-language study of four widely adopted stacks using current runtime versions. We emphasize: (a) methodological fairness (shared schema, identical logic paths); (b) isolation of worker-level performance by minimizing ORMs and higher abstractions; and (c) transparent raw data availability via CSV exports.

2. Related Work

This investigation sits at the intersection of (a) background job frameworks and their queue semantics and (b) language/runtime concurrency models. On the Ruby side, Sidekiq is a widely-used Redis-backed job processor that runs many jobs concurrently using threads in a single process [1], and its public documentation specifies the on-wire Redis job payload shape used by Sidekiq-compatible consumers [2]. Rails’ Active Job provides a standardized interface over multiple queue backends, including Sidekiq, and frames job execution as work performed off the request/response thread [3].

On the Python side, Celery is a distributed task queue with configurable worker concurrency and broker backends [4]. In this repository, a small HTTP enqueue bridge is implemented using Flask [5] to keep Rails’ enqueue path consistent while still executing the Python workload through Celery.

For the polyglot workers, the queue mechanism uses Redis lists with blocking pop semantics; Redis documents BRPOP as a blocking list pop primitive, which underlies the “one-job-at-a-time” consumer loops used here [6]. At the runtime level, Node.js’ official documentation describes the event loop and phases that govern execution of asynchronous callbacks [7], while CPython documents the Global Interpreter Lock (GIL) and its implications for parallel execution of Python bytecode [8]. Go’s official documentation provides the language reference entry point and further guidance on performance topics such as garbage collection [9]. Finally, the shared persistence layer is PostgreSQL, and its documentation provides authoritative definitions for SQL behavior and operational assumptions [10]. Container orchestration is implemented via Docker Compose, for which the official docs define the model used to reproduce the multi-container environment [11].

3. Experimental Design

3.1 Technologies & Versions

- Ruby / Rails:
 - Rails 8.0.2.1
 - Sidekiq 8.0.7
 - Ruby 3.4.5 (as used for this investigation)
- Go worker:
 - Go 1.22 (from `go.mod` and the container build stage)
 - Sidekiq-compatible Redis consumer (not `goworker`)
- Python worker:
 - Python 3.12 (container)
 - Celery 5.4.0, Flask 3.0.3, redis 5.0.4, pycpg2-binary 2.9.9
- Node worker:
 - Node.js 24.x (container; see Dockerfile) / pg 8.11.3
 - Sidekiq-compatible Redis consumer
- Data stores:
 - PostgreSQL server (Docker) 17.6
 - Redis (Docker image) 8.2

3.2 Hardware & Environment

- CPU: Apple M2 Pro
- Cores: 10 physical / 10 logical
- RAM: 16 GiB (17,179,869,184 bytes)
- macOS: 15.6.1 (Build 24G90)
- Kernel/arch: Darwin 24.6.0 (arm64)
- Versions: see Section 3.1 (pinned by manifests / container images)
- Containerization: Docker / Compose for consistent runtime isolation.
- Redis: Shared instance for all queue consumers.
- PostgreSQL: Single shared database (PostgreSQL 17.6 in Docker).

3.3 Benchmark Task

For each page size (`per_page`), workers compute fundamental statistics (min, max, mean, median, quartiles) over a slice of survey values and record result metrics (duration, memory). Computations avoid language-specific collection utilities beyond basic loops and array operations. No caching. All statistical routines executed per task invocation.

3.4 Data Preparation

- Base dataset: synthetic numeric survey-like values stored in `samples.value` (float).
- Dataset size is deterministic and controlled by environment variables consumed by `bin/rails db:seed`:
 - `ROWS` (default 100000): number of `samples` rows to generate.
 - `SEED` (default 123): RNG seed.
 - `DIST` (default `survey`): one of:
 - * `uniform`: `rand()` in $[0, 1]$.
 - * `normal`: Box–Muller transform with $\mu = 0.5$, $\sigma = 0.15$, clamped to $[0, 1]$.
 - * `survey`: discrete-like buckets 1.5 with mild noise ± 0.05 ; bucket weights are 10%/25%/35%/20%/10% for 1/2/3/4/5. This yields values approximately in $[0.95, 5.05]$.
- The seed is idempotent: it deletes all rows in `samples` and reinserts deterministically in batches.
- Task set: sequence of `per_page` sizes (e.g. 1, 10, 25, 50, 100, 250, 500, 1000, 10000).
- Repetitions: each (language, `per_page`) pair executed N times (N configurable; default 30) for variance estimation.

3.5 Metrics

- Duration: Wall-clock time for the statistical computation phase only, measured inside the worker. The timer starts after the data window has been fetched from PostgreSQL and stops when `calculateStatistics` returns. Redis dequeue time, SQL fetch time, and result write time are all excluded. All four workers implement timing identically in this respect.
- Memory: Peak OS-level RSS (resident set size) sampled at 10 ms intervals during job execution, using the same methodology across all four runtimes:
 - **Linux**: `/proc/self/statm` (resident pages $\times$ page size), falling back to `VmRSS` from `/proc/self/status`.
 - **macOS**: `ps -o rss=` for the worker process.

- All four workers implement this same polling approach — a background sampler thread/goroutine reads RSS every 10 ms while the job runs and records the peak observed value. Because workers are long-running services sharing process memory across jobs, the peak RSS reflects the high-water mark of the process working set during that job, not the marginal allocation of a single job in isolation. Comparisons across runtimes should be interpreted in this context (see Section 11).
- Distribution statistics: Captured post-run for plotting (min, q1, median, q3, max).

3.6 Fairness Controls

- Identical SQL queries; minimal driver-level differences.
- Disable extraneous logging where possible.
- Concurrency is intentionally minimized to reduce cross-run interference:
 - Sidekiq runs with `concurrency: 1`.
 - Celery runs with `--concurrency=1` and `--prefetch-multiplier=1` by default in Compose.
 - Go and Node services process one job at a time in a single blocking BRPOP loop.
- Scheduling is configurable; the default for article generation is `SCHEDULE=serial_by_handler` with `WAIT=true`, which:
 - enqueues/runs all tasks for one handler type at a time, then waits for completion before moving to the next handler
 - avoids overlap between handler types (reducing DB/Redis contention and I/O noise)
- Queue isolation: handler types use dedicated queues where applicable (`go`, `node`, `python`), while Ruby uses Sidekiq's `default`.
- Warm-up: The harness does not discard warm-up runs; all configured runs are included in exports. Workers are persistent long-running services (not process-per-job), so JIT warm-up and process startup costs are not repeated per job. The first job in each handler's batch may incur one-time connection establishment or initial allocation overhead; with `RUNS=30` and serial scheduling this is diluted across the series and does not materially affect the median.

4. Implementation Overview

4.1 Rails / Sidekiq

UI orchestration of task creation, selection, export endpoints (`durations_csv`, `memory_csv`, long-format multi-task endpoints). Sidekiq worker executes statistical aggregation using arrays and manual loop-based calculations.

4.2 Go worker (Redis consumer)

Worker consumes Sidekiq-format jobs from Redis, performs direct SQL range queries, computes stats with simple slice operations, and writes results back to PostgreSQL.

4.3 Python / Celery

Celery tasks triggered via Redis broker, using `psycpg` or equivalent for direct queries, manual statistics functions for parity.

4.4 Node.js Worker

Sidekiq-compatible JSON job consumption from Redis; `pg` module with pooled connections. Statistics computed with pure JS arrays; memory instrumentation via process reporting (e.g., `process.memoryUsage()`).

4.5 Shared Schema

Tables: `tasks`, `handlers`, `test_runs`, `test_results`, `statistics`. Raw series persisted in `test_results`; aggregated stats in `statistics` after handler completion.

5. Data Collection & Export

5.1 Storage

Raw durations and memory values per test result stored with timestamps. Aggregated statistical rows saved once handler series is complete.

5.2 CSV Export Facilities

- Per-task wide format: index + one column per handler type (duration or memory). Provided via `TaskSeriesCsvExporter`.
- Multi-task long format: `task`, `handler_type`, `index`, `metric` (separate services for durations and memory). Provided via `SelectedTasksDurationCsvExporter` and `SelectedTasksMemoryCsvExporter`.

5.3 JSON / API

This investigation uses CSV exports as the canonical interchange format for analysis and figure generation.

6. Visualization

D3 boxplot rendering for each handler type across `per_page` values (duration and memory). Figures are generated from the exported CSVs and written to:

- `figures/figure_duration_boxplots.svg`
- `figures/figure_memory_boxplots.svg`

7. Statistical Treatment

- Central tendency: Median is the primary comparison metric, preferred over mean for resilience to right-skewed distributions that arise from occasional GC pauses, scheduler delays, and Redis round-trip jitter.
- Spread: IQR ($q_1 - q_3$) reported alongside each median. Rather than applying formal significance tests, we assess separation by checking whether IQRs overlap between runtime pairs at each workload. At `per_page` ≥ 1000 , the IQRs of every pairwise comparison are non-overlapping — with one exception: Ruby and Python overlap at `per_page=10000` and `per_page=100000`, consistent with the conclusion in Section 9 that they perform comparably at scale. All other pairs have a clear gap between one runtime's q_3 and the next runtime's q_1 , supporting the rank ordering claims without ambiguity.

- Outlier handling: No trimming; whiskers in the boxplot figures show the full observed range per (handler, `per_page`).
- Confidence intervals: Not reported. IQR non-overlap at the relevant workloads makes the ordering unambiguous for all comparisons except Ruby vs Python, which are treated as comparable.

8. Results

This section reports empirical results from the deterministic pipeline (`RUNS=30` per (handler, `per_page`)). The canonical numeric tables (q1/median/q3) are generated from the exported long-format CSVs and recorded in:

- `data/results_summary.md`

Figures are generated from the same CSV inputs and written to:

- `figures/figure_duration_boxplots.svg`
- `figures/figure_memory_boxplots.svg`

8.1 Representative medians (q1/median/q3)

The complete per-workload table is in `data/results_summary.md`. Key representative points:

Duration (seconds):

<code>per_page</code>	Go	Ruby	Python	Node
1	0.000000	0.000009	0.000009	0.000008
1,000	0.000047	0.000173	0.000156	0.000322
	[0.000044, 0.000051]	[0.000166, 0.000180]	[0.000153, 0.000161]	[0.000223, 0.000494]
10,000	0.000605	0.002182	0.001704	0.003334
	[0.000597, 0.000610]	[0.001963, 0.002924]	[0.001688, 0.001740]	[0.002346, 0.005898]
100,000	0.007872	0.022576	0.023818	0.058154
	[0.007558, 0.009132]	[0.022116, 0.024303]	[0.020015, 0.026228]	[0.049526, 0.069152]

Memory (bytes):

<code>per_page</code>	Go	Ruby	Python	Node
1	7,892,992	122,269,696	46,137,344	103,964,672
10,000	10,747,904	147,914,752	50,298,880	148,340,736
100,000	11,534,336	165,675,008	94,568,448	228,392,960

Note: quartiles are computed from the sorted per-(task, handler) series using linear interpolation between adjacent sample points. Memory values are OS-level RSS for the worker process at job completion (see Section 3.5).

9. Discussion

9.1 Duration: Go leads at all scales

Go is the fastest handler at every `per_page` value by a consistent margin. At `per_page=100000`, Go’s median duration is 7.87 ms, compared to 22.6 ms for Ruby, 23.8 ms for Python, and 58.2 ms for Node — ratios of approximately $2.9\times$, $3.0\times$, and $7.4\times$ respectively. The advantage is not an artifact of a single workload: it holds from `per_page=100` through `per_page=100000`, and the rank ordering `Go < Ruby ~ Python < Node` is stable across all 30 runs at those sizes.

The primary explanation is Go’s compiled execution model. The benchmark tasks consist of a SQL fetch followed by iterative statistics computation over an in-memory slice. In Go, both operations execute as native machine instructions with no interpreter overhead and minimal per-iteration allocation. Ruby and Python execute the same logic through an interpreter (MRI / CPython), which imposes consistent per-operation overhead. At `per_page=1000`, Ruby (0.173 ms) and Python (0.156 ms) are nearly identical — unsurprising given the same algorithmic structure and similar interpreter overheads.

9.2 Node.js: high variance at intermediate workloads

Node.js shows the widest IQR at `per_page=250–1000`, where its q1–q3 spread can be $3\text{--}5\times$ wider than Ruby or Python at the same workload. At `per_page=250`, the Node IQR is [0.000088, 0.000242] s against Ruby’s [0.000039, 0.000066] s. This spread narrows at the largest workload (`per_page=100000`), where computation dominates over scheduling noise.

The likely cause is the interaction between Node’s event loop and the blocking BRPOP consumer. Although the worker uses a synchronous-style blocking pop, the surrounding event loop introduces variability in when the job’s async DB callbacks complete relative to the scheduler’s next tick. At small-to-medium workloads where the computation phase is short (sub-millisecond), this scheduling jitter is a larger fraction of total job time. At `per_page=100000` it is diluted by the ~ 58 ms computation.

This makes Node a poor fit for CPU-bound background tasks where latency consistency matters — not because it is universally slow, but because its timing distribution is less predictable than the alternatives at intermediate workload sizes.

9.3 Ruby and Python: comparable at scale

Ruby and Python converge at large workloads: at `per_page=100000`, Ruby median is 22.6 ms and Python is 23.8 ms — a 5% difference — and their IQRs overlap ([0.022116, 0.024303] vs [0.020015, 0.026228]), meaning the distributions cannot be separated by IQR alone. This is the one pair in this study where no ordering claim is warranted; they should be treated as equivalent for practical purposes at this scale.

At small workloads (`per_page < 50`), the ordering is noisier — duration is dominated by queue round-trip time and DB query overhead rather than computation — and differences between run-times at these sizes should not be over-interpreted.

9.4 Memory: Go’s footprint is categorically different

Go’s memory footprint is substantially lower than all other runtimes. At `per_page=100000`, Go’s process RSS is ~ 11.5 MB, compared to ~ 166 MB for Ruby, ~ 94.6 MB for Python, and ~ 228 MB

for Node. These are not marginal differences: Go uses approximately 14× less memory than Ruby and 20× less than Node at the same workload.

The explanation lies in each runtime’s baseline process overhead. Ruby’s MRI runtime and Sidekiq’s threading model load a substantial amount of code and object space before any job runs. Node’s V8 heap adds a significant baseline even with a minimal worker. Python with Celery sits in between. Go’s binary is self-contained and its runtime is lean by design.

Memory grows modestly with workload for all runtimes. Go’s growth from `per_page=1` (7.9 MB) to `per_page=100000` (11.5 MB) reflects the in-memory slice allocation for larger datasets. Ruby grows from ~122 MB to ~166 MB. Node’s growth is more pronounced: from ~104 MB at `per_page=1` to ~228 MB at `per_page=100000`, suggesting that V8 heap growth is not fully reclaimed between jobs in this long-running service configuration.

9.5 Practical implications

For teams choosing a stack for CPU-bound background job workloads:

- **Go** offers the best duration and memory profile across all workload sizes. It is the best choice when latency and memory efficiency are primary constraints.
- **Ruby and Python** perform comparably at scale and are reasonable choices when team familiarity or existing infrastructure outweigh raw performance. The ~3× duration overhead vs Go is consistent and predictable.
- **Node.js** is not well-suited to CPU-bound batch tasks in this configuration. Its median duration is 2.5× Ruby at large workloads, and its IQR is wide at intermediate sizes, making tail latency harder to bound.

These conclusions hold under the conditions of this benchmark: single-threaded workers, serial scheduling, a shared PostgreSQL backend, and purely in-memory statistical computation. Workloads with significant I/O wait, network latency, or parallelism may produce different relative orderings.

10. Reproducibility

This repository is designed to be reproducible with Docker Compose, with a single canonical path that (a) seeds data deterministically, (b) runs a fixed benchmark schedule, and (c) exports CSVs and renders figures. The reproducibility target is regenerated benchmark conditions and statistically comparable results, not byte-identical exported artifacts across live reruns.

For archival publication, the repository is packaged as a single monorepo-style snapshot: `benchmark_ui/`, `golang_worker/`, `node_worker/`, and `python_worker/` are stored as ordinary directories in the artifact rather than as external submodules.

CSV export endpoints (if you prefer fetching via HTTP rather than using the tasks below): - Per-task wide format: `/tasks/:id/durations_csv`, `/tasks/:id/memory_csv` - Multi-task long format (selected tasks): `/durations/selected_csv`, `/memory/selected_csv`

10.1 Recommended: Docker Compose (canonical, end-to-end)

From `rails_8/`:


```
# Build images
```

```
docker compose build
```

```
# Start infra + workers + Rails UI + Sidekiq
```

```
docker compose up -d postgres redis
```

```
docker compose up -d benchmark_ui sidekiq
```

```
docker compose up -d golang_worker node_worker python_worker python_worker_api
```

Run the fixed benchmark pipeline (seed + create tasks + enqueue runs) inside the Rails container:

```
docker compose run --rm benchmark_ui bin/rails db:prepare
```

```
docker compose run --rm benchmark_ui bash -lc '  
ROWS=100000 SEED=123 DIST=survey \  
PER_PAGES=1,10,25,50,100,250,500,1000,10000,100000 RUNS=30 PAGE=1 \  
HANDLERS=ruby,go,python,node MODE=enqueue SCHEDULE=serial_by_handler WAIT=true \  
bin/rails article:generate_all  
'
```

For a clean clone, `db:prepare` must run before `article:generate_all`; the benchmark command is executed through `bash -lc`, so it does not use the server entrypoint path that auto-prepares the database.

After workers finish, export the canonical long-format CSVs and render the static figures:

```
docker compose run --rm benchmark_ui bash -lc 'bin/rails article:export_selected_csv'
```

```
docker compose run --rm benchmark_ui bash -lc 'bin/rails article:generate_figures'
```

Primary outputs (written under `benchmark_ui/docs/`): - `data/durations_selected.csv` - `data/memory_selected.csv` - `data/results_summary.md` - `figures/figure_duration_boxplots.svg` - `figures/figure_memory_boxplots.svg`

These files are archival outputs for a specific validated run. When the pipeline is rerun on a real machine, exact file hashes may change because the measured runtime and memory series vary slightly from run to run.

Validated end-to-end run commit: `72cd7ce608a10d3f6f3ba51464da1469f87a6a18`.

Teardown:

```
docker compose down -v
```

10.2 Optional: local Rails (non-Docker)

If you run Rails on the host instead of inside Compose, the minimum steps are: - Ensure PostgreSQL and Redis are running (via Compose or locally). - Seed deterministically:

```
```bash  
ROWS=100000 SEED=123 DIST=survey bin/rails db:seed
```
```

- Start Rails UI: `bin/rails server`.
- Enqueue benchmark runs from the UI or via API; run workers separately.

- Export CSVs and render figures: `bin/rails article:export_selected_csv` and `bin/rails article:generate_figures`.

10.3 Optional: run tests (Docker)

From `rails_8/`:

```
docker compose run --rm benchmark_ui bundle exec rspec
docker compose run --rm golang_worker_test
docker compose run --rm node_worker node --test
docker compose run --rm python_worker python -m unittest discover -s tests -p 'test_*.py'
```

10.4 Environment versions capture

- Record outputs of `ruby -v`, `bundle -v`, `node -v`, `go version`, `python --version`, `psql --version`, `redis-server --version`.
- Version pinning is captured in `Gemfile`, `go.mod`, `requirements.txt`, and `package.json`.

10.5 Interpreting reruns

- SEED controls deterministic synthetic input generation.
- RUNS repeats the same benchmark condition to estimate runtime and memory distributions.
- On a real machine, scheduler timing, garbage collection, queue timing, and long-lived worker state will shift some observed measurements.
- Reproducibility should therefore be assessed by configuration parity and statistical comparability, not exact output hashes.

10.6 Rerun acceptance policy

- Exact-match requirements:
 - same benchmark commit or release tag
 - same benchmark parameters: `ROWS`, `SEED`, `DIST`, `PER_PAGES`, `RUNS`, `PAGE`, `HANDLERS`, `MODE`, `SCHEDULE`, `WAIT`
 - same worker-concurrency constraints: Sidekiq concurrency 1, Celery concurrency 1, Celery prefetch multiplier 1, and one-job Go/Node loops
 - regeneration of the same canonical output set under `benchmark_ui/docs/`
- Statistical-match requirements:
 - for `per_page` ≥ 100 , each handler's duration median should remain within a 0.5x to 2.0x band of the archival run median — except Node.js at `per_page` < 1000 , where the band is 0.25x to 4.0x due to its documented high scheduling variance at small workloads
 - for `per_page` ≥ 100 , each handler's memory median should remain within a 0.8x to 1.25x band of the archival run median
 - for `per_page` = 1000, 10000, and 100000, the qualitative ordering of handler medians should be preserved — except that Ruby and Python are treated as an unordered pair, since their IQRs overlap at large workloads and their relative order is within noise
 - no handler should exhibit an order-of-magnitude shift at any workload without an explained code or environment change
 - rerun reports should capture the machine and environment used
- Non-goal:

- byte-identical CSV, Markdown, and SVG outputs are not required for live reruns on a real machine

The smallest workloads (`per_page < 100`) are excluded from the numeric duration threshold because microsecond-scale timings are dominated more easily by scheduler and timer noise.

Fresh-clone rerun validation (2026-06-02): - Commit: 3778b5d2ea6c3ef1f158719907895b091b5a336e
 - Environment: Apple M2 Pro, macOS, Docker Compose (fresh clone from GitHub) - Result: **PASS** — all handlers within ratio bands; no order violations outside the Ruby/Python unordered pair exception - Automated check: `comparison_run_id=9` in the benchmark UI database

10.7 Notes on containerized services

- Compose defaults:
 - Postgres: `postgres:5432 database benchmark_development` with `postgres/postgres`
 - Redis: `redis://redis:6379/0`
 - Rails runs in development mode in Compose to avoid requiring a master key.
- Python enqueue URL:
 - If Rails runs on the host, use `http://127.0.0.1:5001/enqueue` (host-published port).
 - If Rails runs in Compose, use `PYTHON_WORKER_URL=http://python_worker_api:5000/enqueue` (service DNS + container port).

11. Limitations & Threats to Validity

Single-machine scope. All results were collected on one Apple M2 Pro under macOS in Docker containers. Performance ratios may differ on Linux bare-metal, cloud instances, or machines with different memory bandwidth and CPU characteristics. The rank ordering `Go < Ruby ~ Python < Node` is expected to be stable across hardware, but the absolute multiples may not transfer directly.

Memory measurement is process-level, not job-marginal. All four workers measure peak RSS using the same `/proc/self/statm → /proc/self/status → ps -o rss=` approach sampled at 10 ms intervals during job execution (see Section 3.5). The measurement captures the high-water mark of the process working set during the job, not the incremental allocation cost of that job alone. Ruby’s baseline Sidekiq process consumes ~120 MB before any work is done; Go’s baseline is ~8 MB. Differences in memory readings at small workloads largely reflect runtime baseline overhead rather than algorithmic allocation. This is the intended measurement for capacity planning purposes, but it does not isolate per-job heap allocation.

Duration measures computation, not end-to-end job latency. The timer runs only over the `calculateStatistics` call — Redis dequeue, SQL fetch, and result write are excluded. This is the intended scope: the benchmark measures computational throughput rather than full pipeline latency. End-to-end job latency would include queue wait time and is not captured.

No warm-up discard. The first job in each handler’s batch may incur one-time overhead (connection establishment, initial memory allocation). With `RUNS=30` and serial scheduling, the first-job effect is diluted and does not materially affect the median, but it contributes to the q1 spread at small workloads.

No cross-machine or temporal validation. Results were collected in a single session. Re-runs on the same machine at different times may shift absolute values due to thermal throttling,

background OS activity, and Docker scheduling. The rerun acceptance policy (Section 10.6) defines the expected statistical stability bounds.

12. Conclusion & Future Work

We supply a transparent multi-language benchmark harness enabling apples-to-apples comparisons for background computational tasks. Future directions: Rust implementation; distributed multi-node scaling; inclusion of JVM-based (e.g., Kotlin + Spring Batch) and Elixir (BEAM) ecosystems; power efficiency metrics.

13. References

1. Sidekiq. “Sidekiq” (README / project overview). <https://github.com/sidekiq/sidekiq> (accessed 2026-05-31).
2. Sidekiq Wiki. “Job Format”. <https://github.com/sidekiq/sidekiq/wiki/Job-Format> (accessed 2026-05-31).
3. Ruby on Rails Guides. “Active Job Basics”. https://guides.rubyonrails.org/active_job_basics.html (accessed 2026-05-31).
4. Celery Documentation. “Celery - Distributed Task Queue”. <https://docs.celeryq.dev/en/stable/> (accessed 2026-05-31).
5. Flask Documentation. “Welcome to Flask”. <https://flask.palletsprojects.com/> (accessed 2026-05-31).
6. Redis Documentation. “BRPOP”. <https://redis.io/commands/brpop/> (accessed 2026-05-31).
7. Node.js Documentation. “The Node.js Event Loop”. <https://nodejs.org/en/learn/asynchronous-work/event-loop-timers-and-nexttick> (accessed 2026-05-31).
8. Python Documentation. “Glossary: global interpreter lock”. <https://docs.python.org/3/glossary.html#term-global-interpreter-lock> (accessed 2026-05-31).
9. Go Documentation. “Documentation” (entry point), and linked performance topics such as “A Guide to the Go Garbage Collector”. <https://go.dev/doc/> (accessed 2026-05-31).
10. PostgreSQL Documentation. “PostgreSQL Documentation (current)”. <https://www.postgresql.org/docs/current/> (accessed 2026-05-31).
11. Docker Documentation. “Docker Compose”. <https://docs.docker.com/compose/> (accessed 2026-05-31).

Repository artifact mapping: - Export services: `TaskSeriesCsvExporter`, `SelectedTasksMemoryCsvExporter`, `SelectedTasksDurationCsvExporter`. - Request specs validate CSV endpoints. - D3 visualization integrated in UI pages for memory and duration.